Software Requirements Specification

for

TripNest

Real-Time Collaborative Travel & Expense Engine

Document Version: 2.1 (17-Table Flagship Architecture)
Target Architecture: Spring Boot 3.x, PostgreSQL 15+, Redis, AWS S3, STOMP WebSockets, React 18
Core Focus: Write-Then-Broadcast Notifications, Distributed Sync, Financial Integrity, Contextual RBAC

Date: August 2026

Contents

1 Introduction & Architectural Scope **3**
1.1 Purpose . . . 3
1.2 System Stack Topology . . . 3

2 Stakeholders & Contextual Actor Matrix **3**
2.1 Actor Taxonomy . . . 3
2.2 Actor → Use Case Mapping . . . 3

3 Functional Requirements (Core 10 Modules) **4**
3.1 01. Authentication & Identity Management . . . 4
3.2 02. Trip Workspace & Membership (RBAC) . . . 4
3.3 03. Day-Wise Itinerary & Optimistic Scheduling . . . 4
3.4 04. Real-Time Distributed Collaboration (STOMP) . . . 4
3.5 05. Multi-Currency Financial Ledger & Split Engine . . . 5
3.6 06. Debt Simplification & Settlement Engine . . . 5
3.7 07. Dual-Pattern S3 Vault (Shared Docs vs. Personal Wallet) . . . 5
3.8 08. Shared Media Gallery (Photos & Videos) . . . 5
3.9 09. External Services & Write-Then-Broadcast Notifications . . . 5
3.10 10. Targeted Audit Trail & Observability . . . 5

4 Database Architecture & Relational Schema **6**
4.1 Entity Relationship Mapping (17 Tables) . . . 6

5 Non-Functional Requirements & Invariants **9**
5.1 Performance & Distributed Invariants . . . 9
5.2 Security & Data Governance . . . 10

6 System Constraints & Assumptions **10**
6.1 Technology Constraints . . . 10
6.2 Authentication Constraints . . . 10
6.3 External Service Constraints . . . 10
6.4 Travel Booking Constraints . . . 11
6.5 Internet Dependency . . . 11
6.6 Real-Time Communication Constraints . . . 11
6.7 File Storage Constraints . . . 11
6.8 Financial Constraints . . . 12
6.9 Currency Constraints . . . 12
6.10 Privacy Constraints . . . 12
6.11 Project Scope Constraints . . . 12
6.12 Assumptions . . . 12
6.13 Explicitly Out of Scope . . . 13

7 External Interface Requirements **13**
7.1 Authentication Interface — Google OAuth 2.0 / OIDC . . . 13
7.2 Maps Interface — Google Maps . . . 13
7.3 Weather Interface — OpenWeather . . . 13
7.4 Currency Exchange Interface . . . 14
7.5 Object Storage Interface — AWS S3 . . . 14
7.6 Email Interface — AWS SES . . . 14
7.7 Redis Interface . . . 14
7.8 PostgreSQL Interface . . . 15
7.9 WebSocket/STOMP Interface . . . 15

7.10 REST API Interface . . . 15
7.11 External Interface Failure Policy . . . 15
7.12 External Interface Security . . . 16

8 API Requirements & Interface Specification 17
8.1 API Standards . . . 17
8.2 REST API Endpoint Specifications . . . 17
8.2.1 1. Authentication APIs (`/api/v1/auth`) . . . 17
8.2.2 2. User Profile APIs (`/api/v1/users`) . . . 17
8.2.3 3. Trip Workspace APIs (`/api/v1/trips`) . . . 18
8.2.4 4. Membership APIs (`/api/v1/trips/{tripId}/members`) . . . 18
8.2.5 5. Itinerary APIs (`/api/v1/trips/{tripId}/itinerary`) . . . 18
8.2.6 6. Expense & Settlement APIs . . . 18
8.2.7 7. Travel Wallet APIs (`/api/v1/wallet`) . . . 19
8.2.8 8. Document & Media S3 APIs . . . 19
8.2.9 9. Chat & Notification APIs . . . 19
8.2.10 10. External Proxy Data APIs . . . 20
8.3 WebSocket STOMP Channel Topology . . . 20

9 ER Diagram & Database Specification 20
9.1 Cardinality Summary . . . 20
9.2 Database Indexing Strategy . . . 22
9.3 Cascade Deletion & Data Retention Policy . . . 24

10 System Architecture & Component Design 24
10.1 Architectural Style: Modular Monolith . . . 24
10.2 Backend Package-by-Feature Layout . . . 24

11 Critical Sequence Diagrams & Workflows 24
11.1 11.1 Create Trip & Assign Owner . . . 24
11.2 11.2 Collaborative Itinerary Update & Optimistic Locking . . . 24
11.3 11.3 Idempotent Expense Logging & Split Allocation . . . 25
11.4 11.4 Two-Phase Direct-to-S3 Upload . . . 25

12 Security Specification 25
12.1 Authentication & Token Management . . . 25
12.2 Contextual Role-Based Access Control (RBAC) . . . 25
12.3 Real-Time & Storage Security . . . 26

13 Testing & Acceptance Criteria 26
13.1 Testing Verification Matrix . . . 26
13.2 Definition of Done (DoD) . . . 26

1 Introduction & Architectural Scope

1.1 Purpose

This document specifies the software requirements for **TripNest**. The system focuses on core distributed backend patterns: **atomic financial ledgers, graph-based debt simplification, multi-user optimistic locking, STOMP WebSocket broadcasts, dual-purpose S3 pre-signed upload security, contextual role-based access control (RBAC), and persistent write-then-broadcast in-app notifications**.

1.2 System Stack Topology

- **Backend Core:** Java 21 / Spring Boot 3.x with Spring Security, Spring Web, and Spring Data JPA.
- **Persistence Layer:** PostgreSQL 15+ managed via Flyway migration scripts.
- **Distributed Caching & In-Memory State:** Redis (Forex rates, Weather responses, Idempotency keys).
- **Real-Time Message Broker:** Spring WebSocket with STOMP over SockJS (User-targeted queues + Trip broadcast topics).
- **Object Storage:** Amazon Web Services (AWS) S3 utilizing pre-signed URLs.
- **Frontend Application:** React 18 with TanStack Query, Tailwind CSS, and HTML5 drag-and-drop.

—

2 Stakeholders & Contextual Actor Matrix

2.1 Actor Taxonomy

- **Global Actors:** `Guest` (Unauthenticated), `User` (Authenticated Session).
- **Contextual Trip Roles:** `Owner` (Full Management/Deletion), `Admin` (Moderator/Manager), `Member` (Collaborator), `Viewer` (Read-Only).
- **External Integration Actors:** `OAuth Provider` (Google), `Maps API` (Google), `Weather API` (OpenWeather), `Exchange Rate API`, `AWS S3`.

2.2 Actor → Use Case Mapping

Core Architectural Workflow	Guest	User	Owner	Admin	Member	Viewer	Integration Provider
User Register / JWT Login			–	–	–	–	Google OAuth
Create Trip & Claim Owner	–		–	–	–	–	–
Invite / Manage Members	–	–			–	–	AWS SES
Day-Wise Itinerary Planning	–	–				Read	STOMP WebSockets
Location Geocoding	–					–	Google Maps
Cached Weather Retrieval	–						OpenWeather + Redis
Multi-Currency Expense Logging	–	–				Read	Redis (Forex Rates)
Debt Settlement Recording	–	–				Read	–
Real-Time Trip Chat	–	–				Read	STOMP WebSockets
Direct S3 Document / Media Upload	–	–				–	AWS S3 Pre-signed

Core Architectural Workflow	Guest	User	Owner	Admin	Member	Viewer	Integration Provider
Private Travel Wallet Management	–		–	–	–	–	AWS S3 Pre-signed
Write-Then-Broadcast Notifications	–						STOMP Queue / PostgreSQL
Targeted Audit Log Inspection	–	–			–	–	PostgreSQL

—

3 Functional Requirements (Core 10 Modules)

3.1 01. Authentication & Identity Management

- **FR-AUTH-001 (Registration & Auth):** The system shall authenticate users via Email/-Password (BCrypt hashed) and Google OAuth 2.0 (OIDC).
- **FR-AUTH-002 (Stateless JWT):** The system shall issue cryptographically signed JWT access tokens (15-min TTL) and rotating refresh tokens (7-day TTL).
- **FR-AUTH-003 (Session Invalidation):** Logout operations shall revoke refresh tokens in PostgreSQL and invalidate active client state.

3.2 02. Trip Workspace & Membership (RBAC)

- **FR-TRIP-001 (Trip Creation):** Authenticated users shall initialize trips with title, destination, dates, base currency, and cover image.
- **FR-TRIP-002 (Single-Owner Guarantee):** The trip creator automatically becomes the sole `OWNER`. Ownership transfer demotes the previous owner to `ADMIN`.
- **FR-TRIP-003 (Tokenized Invitations):** The system shall generate expiring cryptographic invite links to allow users to join the workspace.
- **FR-TRIP-004 (Contextual RBAC):** Operations are guarded by hierarchical permissions: OWNER > ADMIN > MEMBER > VIEWER.

3.3 03. Day-Wise Itinerary & Optimistic Scheduling

- **FR-ITIN-001 (Day Management):** Authorized members shall create and manage day containers mapped to trip dates.
- **FR-ITIN-002 (Activity Ordering via LexoRank):** Activities within a day shall maintain dynamic sequences using fractional lexicographical indexing (LexoRank) to avoid bulk index writes.
- **FR-ITIN-003 (Optimistic Locking):** Concurrent updates to activities shall use JPA entity versioning (`@Version`) to prevent race overwrites.

3.4 04. Real-Time Distributed Collaboration (STOMP)

- **FR-REAL-001 (Channel Subscription):** Authenticated clients establish secure WebSocket connections at `/ws-trip` and subscribe to destination topics.
- **FR-REAL-002 (Channel Interception):** A custom STOMP `ChannelInterceptor` shall validate JWT tokens and trip membership before authorizing topic subscriptions.
- **FR-REAL-003 (Live Mutation Broadcast):** Activity movements, deletions, and chat frames shall broadcast instantly ($\leq$ 100ms) to connected trip members.

3.5 05. Multi-Currency Financial Ledger & Split Engine

- **FR-EXP-001 (Transactional Creation):** Expense persistence, split rows, and balance updates must execute inside a single atomic `@Transactional` boundary.
- **FR-EXP-002 (Idempotency):** Expense creation and settlement requests must enforce an `Idempotency-Key` HTTP header stored in Redis to prevent double execution.
- **FR-EXP-003 (Split Verification):** The system shall validate that the sum of individual splits strictly matches the total expense amount ($\sum$ Split = Total).
- **FR-EXP-004 (Redis Forex Normalization):** Foreign currency amounts are converted to the trip's base currency using exchange rates cached in Redis (24h TTL).

3.6 06. Debt Simplification & Settlement Engine

- **FR-SETTLE-001 (Greedy Simplification Algorithm):** The system shall compute a minimal transaction graph from pairwise net balances to minimize the total payments required across the group.
- **FR-SETTLE-002 (Settlement Ledger):** Settling payments records an immutable transaction row in `settlements` and updates the outstanding debt ledger.

3.7 07. Dual-Pattern S3 Vault (Shared Docs vs. Personal Wallet)

- **FR-S3-001 (Two-Phase Pre-signed Upload):** Clients request an expiring S3 `PUT` URL, upload directly to AWS S3, and notify the backend to commit metadata.
- **FR-S3-002 (Shared Documents vs. Private Wallet):**
  - *Shared Vault:* Document metadata is attached to `trip_id`; accessible to all trip members.
  - *Personal Wallet:* Items are bound to `user_id`; strictly isolated from other members unless explicitly shared.
- **FR-S3-003 (Zero Public Buckets):** S3 buckets prohibit public access; downloads require time-limited (15-min) pre-signed `GET` URLs.

3.8 08. Shared Media Gallery (Photos & Videos)

- **FR-MEDIA-001 (Direct Media Ingestion):** Members upload trip memories using the S3 pre-signed upload pattern.
- **FR-MEDIA-002 (Chronological Gallery):** Shared photos/videos are organized chronologically with author tracking.

3.9 09. External Services & Write-Then-Broadcast Notifications

- **FR-EXT-001 (Geocoding & Location Pinning):** Google Places API autocompletes destination queries and saves latitude/longitude coordinates to activities.
- **FR-EXT-002 (Cached Weather Forecasts):** Current weather and 5-day forecasts are fetched from OpenWeatherMap, cached in Redis for 3 hours, and degraded gracefully on failure.
- **FR-EXT-003 (Write-Then-Broadcast Notification Invariant):** When an event occurs (e.g., expense split assigned, trip invite issued), the backend shall atomically persist a notification row to `notifications` and dispatch a targeted STOMP frame to `/user/{userId}/queue/notifications`.
- **FR-EXT-004 (Notification Consumption):** Users shall fetch paginated notifications via `GET /api/v1/notifications` and mark them read via `PATCH /api/v1/notifications/{id}/read`.

3.10 10. Targeted Audit Trail & Observability

- **FR-AUDIT-001 (Selective Event Logging):** The system shall record immutable audit records strictly for high-impact events: `TRIP_CREATED`, `MEMBER_REMOVED`, `ROLE_CHANGED`, and `EXPENSE_CREATED`.

- **FR-AUDIT-002 (Structured Diagnostics):** Correlation IDs (`X-Request-ID`) are propagated through SLF4J logs and RFC 7807 Problem Detail exception responses.

—

4 Database Architecture & Relational Schema

4.1 Entity Relationship Mapping (17 Tables)

Table Name	Attributes & Constraints	Primary Key / FKs
users	email (UNIQUE), password_hash, full_name, avatar_url, status, created_at, updated_at	id (UUID)
oauth_accounts	provider, provider_user_id (UNIQUE with provider), linked_at	id (UUID), user_id (FK)
refresh_tokens	token_hash (UNIQUE), expires_at, is_revoked, created_at	id (UUID), user_id (FK)
trips	title, description, destination, start_date, end_date, base_currency, cover_image_url, created_at	id (UUID), owner_id (FK)
trip_members		id (UUID), trip_id (FK), user_id (FK)

Table Name Attributes & Constraints	Primary Key / FKs
role (OWNER, ADMIN, MEMBER, VIEWER), joined_at, CONSTRAINT: UNIQUE(trip_id, user_id)	
trip_invitations	id (UUID), trip_id (FK), invited_by (FK)
token_hash (UNIQUE), role, expires_at, usage_limit, usage_count, is_revoked, created_at	
itinerary_days	id (UUID), trip_id (FK)
day_number (INT), date (DATE), title, notes, CONSTRAINT: UNIQUE(trip_id, day_number)	
activities	id (UUID), day_id (FK), created_by (FK)
title, location_name, latitude, longitude, start_time, end_time, order_index (VARCHAR), cost, version (INT)	
expenses	id (UUID), trip_id (FK), paid_by (FK)

Table Name Attributes & Constraints	Primary Key / FKs
amount (NUMERIC(12,2)), currency (ISO-4217), description, category, expense_date, receipt_s3_key, created_at	
expense_splits	id (UUID), expense_id (FK), user_id (FK)
owed_amount (NUMERIC(12,2)), is_settled (BOOL), CONSTRAINT: UNIQUE(expense_id, user_id)	
settlements	id (UUID), trip_id (FK), payer_id (FK), receiver_id (FK)
amount (NUMERIC(12,2)), currency, settlement_date, created_at	
wallet_items	id (UUID), user_id (FK), trip_id (FK, Nullable)
title, item_type (TICKET, CONFIRMATION, ID), s3_key, is_shared (BOOL), created_at	
documents	id (UUID), trip_id (FK), uploaded_by (FK)
file_name, file_type, file_size_bytes, s3_key (UNIQUE), created_at	
trip_media	id (UUID), trip_id (FK), uploaded_by (FK)

Table Name Attributes & Constraints	Primary Key / FKs
s3_key (UNIQUE), media_type (PHOTO, VIDEO), caption, created_at	
chat_messages	id (UUID), trip_id (FK), sender_id (FK)
message_content (TEXT), created_at	
notifications	id (UUID), user_id (FK), trip_id (FK, Nullable)
type (VARCHAR), title, message, is_read (BOOL DEFAULT FALSE), action_url, created_at	
audit_logs	id (UUID), actor_id (FK, Nullable), trip_id (FK, Nullable)
action (VARCHAR), entity_type, entity_id, metadata (JSONB), client_ip, created_at	

—

5 Non-Functional Requirements & Invariants

5.1 Performance & Distributed Invariants

- **NFR-PERF-001:** Standard REST endpoints respond within $\leq$ 200ms at the 95th percentile under standard load.
- **NFR-REAL-001:** WebSocket STOMP broadcast latency shall not exceed 100ms across topics and user notification queues.
- **NFR-REL-001 (Atomic Boundaries):** All expense mutations, split assignments, and settlement updates execute under `@Transactional(isolation = Isolation.READ_COMMITTED)`.
- **NFR-CACHE-001 (Redis In-Memory Layer):** Exchange rates (24h TTL), weather forecasts (3h TTL), and Idempotency keys (24h TTL) reside strictly in Redis to preserve DB performance.

5.2 Security & Data Governance

- **NFR-SEC-001:** Passwords hashed with BCrypt (cost factor 12).
- **NFR-SEC-002:** S3 buckets block all direct public access; file retrieval uses 15-minute pre-signed `GET` URLs.
- **NFR-SEC-003:** Method-level security (`@PreAuthorize`) strictly isolates contextual trip data and personal wallets.
- **NFR-SEC-004:** Global exception handler emits RFC 7807 Problem Detail structures:

```
{
  "timestamp": "2026-08-18T16:55:00Z",
  "status": 400,
  "error": "BAD_REQUEST",
  "code": "INVALID_SPLIT_SUM",
  "message": "The sum of splits does not equal the expense amount",
  "path": "/api/v1/trips/{id}/expenses"
}
```

6 System Constraints & Assumptions

6.1 Technology Constraints

- **CON-TECH-001 (Backend Technology):** The backend shall be implemented using Java 21 and Spring Boot 3.x with Spring Security, Spring Web, and Spring Data JPA[span$_1$]($start_span$)[$span_1$](end_span
- **CON-TECH-002 (Relational Database):** PostgreSQL 15+ shall be used as the primary persistent relational database[span$_2$]($start_span$)[$span_2$](end_span).
- **CON-TECH-003 (Database Migrations):** Database schema changes shall be managed through version-controlled Flyway migration scripts[span$_3$]($start_span$)[$span_3$](end_span).
- **CON-TECH-004 (Distributed Cache):** Redis shall be used for transient distributed state including exchange-rate data, weather responses, and idempotency keys[span$_4$]($start_span$)[$span_4$](end_span).
- **CON-TECH-005 (Real-Time Communication):** Real-time collaboration shall use STOMP over WebSocket with the configured `/ws-trip` endpoint[span$_5$]($start_span$)[$span_5$](end_span).
- **CON-TECH-006 (Object Storage):** AWS S3 shall be used for uploaded documents, photos, videos, and wallet files[span$_6$]($start_span$)[$span_6$](end_span).
- **CON-TECH-007 (Frontend):** The primary client shall be implemented using React 18[span$_7$]($start_span$)[

6.2 Authentication Constraints

- **CON-AUTH-001:** Protected functionality shall require an authenticated user session[span$_8$]($start_span$)[sp
- **CON-AUTH-002:** Email/password authentication shall use BCrypt-hashed passwords[span$_9$]($start_span$)[s
- **CON-AUTH-003:** Google OAuth 2.0/OIDC functionality shall depend on the availability of the configured OAuth provider[span$_10$]($start_span$)[$span_10$](end_span).
- **CON-AUTH-004:** JWT access tokens and refresh tokens shall be managed by the backend[span$_11$]($start_span$
- **CON-AUTH-005:** Authentication credentials and token secrets shall not be exposed to the frontend or stored in source code[span$_12$]($start_span$)[$span_12$](end_span).

6.3 External Service Constraints

- **CON-EXT-001:** Availability of external services (Google Identity, Google Maps, OpenWeatherMap, ExchangeRate-API, AWS S3, AWS SES) is outside TripNest's direct control[span$_13$]($start_span$)[$span$

- **CON-EXT-002:** External services may impose rate limits[span$_1$4]($start_span$)[$span_1$4](end_span).
- **CON-EXT-003:** External service responses may change or become temporarily unavailable[span$_1$5]($start_sp$
- **CON-EXT-004:** External API credentials shall be stored securely on the backend[span$_1$6]($start_span$)[$span_1$
- **CON-EXT-005:** Failure of a non-critical external service shall not make unrelated TripNest functionality unavailable[span$_1$7]($start_span$)[$span_1$7](end_span).

6.4 Travel Booking Constraints

- **CON-BOOK-001:** TripNest shall not process actual travel-provider payments in the current scope.
- **CON-BOOK-002:** Users shall be redirected to the external travel provider to complete the actual booking.
- **CON-BOOK-003:** TripNest shall not guarantee availability, pricing, cancellation policies, or refunds provided by external travel providers.
- **CON-BOOK-004:** Travel-related records stored by TripNest shall represent information available to the user/system and shall not replace the external provider's authoritative booking system.

6.5 Internet Dependency

- **CON-NET-001:** Normal TripNest operation shall require network connectivity.
- **CON-NET-002:** Maps, weather, exchange-rate, OAuth, S3, and notification integrations depend on network connectivity[span$_1$8]($start_span$)[$span_1$8](end_span).
- **CON-NET-003:** Temporary failure of an external service shall be handled without corrupting TripNest's persistent data[span$_1$9]($start_span$)[$span_1$9](end_span).
- **CON-NET-004:** The current scope does not guarantee full offline synchronization.

6.6 Real-Time Communication Constraints

- **CON-REAL-001:** Real-time collaboration requires an active WebSocket connection[span$_2$0]($start_span$)[spa
- **CON-REAL-002:** Disconnected clients may not receive broadcast events while disconnected[span$_2$1]($start_s$
- **CON-REAL-003:** After reconnection, the client shall retrieve the authoritative current state from the backend via REST[span$_2$2]($start_span$)[$span_2$2](end_span).
- **CON-REAL-004:** WebSocket subscriptions shall be subject to the same contextual trip authorization principles as REST resources[span$_2$3]($start_span$)[$span_2$3](end_span).
- **CON-REAL-005:** If multiple backend instances are deployed, additional distributed messaging infrastructure (e.g., Redis Pub/Sub) may be required for cross-instance WebSocket synchronization.

6.7 File Storage Constraints

- **CON-S3-001:** TripNest shall not store large binary files directly in PostgreSQL[span$_2$4]($start_span$)[$span_2$4](
- **CON-S3-002:** Documents, photos, videos, and wallet files shall be stored in AWS S3[span$_2$5]($start_span$)[spa
- **CON-S3-003:** PostgreSQL shall store file metadata and S3 object references[span$_2$6]($start_span$)[$span_2$6](en
- **CON-S3-004:** S3 buckets shall not allow unrestricted public access[span$_2$7]($start_span$)[$span_2$7](end_span).

- **CON-S3-005:** File downloads shall use time-limited authorized access (15-min pre-signed URLs)[span$_2$8]($start_span$)[$span_2$8](end_span).

6.8 Financial Constraints

- **CON-FIN-001:** TripNest shall maintain expense and settlement records but shall not act as a financial institution[span$_2$9]($start_span$)[$span_2$9](end_span).
- **CON-FIN-002:** TripNest shall not hold user funds.
- **CON-FIN-003:** TripNest shall not directly transfer money between users[span$_3$0]($start_span$)[$span_3$0](end_sp

- **CON-FIN-004:** Expense and settlement mutations shall preserve financial consistency through transactional processing[span$_3$1]($start_span$)[$span_3$1](end_span).

6.9 Currency Constraints

- **CON-CUR-001:** Expenses shall retain their original currency and amount[span$_3$2]($start_span$)[$span_3$2](end_s

- **CON-CUR-002:** Foreign-currency normalization shall use exchange-rate data available through the configured exchange-rate provider[span$_3$3]($start_span$)[$span_3$3](end_span).
- **CON-CUR-003:** Exchange-rate data may be cached in Redis according to the defined TTL (24 hours)[span$_3$4]($start_span$)[$span_3$4](end_span).

6.10 Privacy Constraints

- **CON-PRIV-001:** Trip membership shall not automatically grant access to a user's private Travel Wallet[span$_3$5]($start_span$)[$span_3$5](end_span).
- **CON-PRIV-002:** Private wallet items shall remain isolated from other trip members unless explicitly shared[span$_3$6]($start_span$)[$span_3$6](end_span).
- **CON-PRIV-003:** Shared trip documents shall be accessible only to authorized trip members[span$_3$7]($start_s$

- **CON-PRIV-004:** Private S3 objects shall not be publicly accessible[span$_3$8]($start_span$)[$span_3$8](end_span).

6.11 Project Scope Constraints

- **CON-SCOPE-001:** The initial implementation shall prioritize the 10 core functional modules defined in the SRS[span$_3$9]($start_span$)[$span_3$9](end_span).
- **CON-SCOPE-002:** Features not represented in the current functional requirements shall not be considered mandatory for the initial implementation.
- **CON-SCOPE-003:** External integrations that require unavailable credentials or commercial approval may be developed using mock/test providers during development.
- **CON-SCOPE-004:** The system shall maintain clear abstraction boundaries around external integrations so that unavailable providers do not block development of the core backend.

6.12 Assumptions

- **ASM-001:** Users have access to a supported modern browser and an internet connection[span$_4$0]($start_span$)[

- **ASM-002:** Users provide valid and accurate trip information.
- **ASM-003:** Users have permission to upload documents and media that they submit to TripNest.
- **ASM-004:** External APIs provide valid responses according to their documented contracts.
- **ASM-005:** The external OAuth provider remains available when OAuth authentication is used[span$_4$1]($start_span$)[$span_4$1](end_span).

- **ASM-006:** AWS S3 remains available for file storage operations[span$_4$2]($start_span$)[$span_4$2](end_span).
- **ASM-007:** Users participating in shared expenses provide accurate payment and split information[span$_4$3](*s*
- **ASM-008:** The external travel provider remains the authoritative source for actual booking status.
- **ASM-009:** Cached weather and exchange-rate information may become stale after its configured TTL[span$_4$4]($start_span$)[$span_4$4](end_span).

6.13 Explicitly Out of Scope

- Direct flight/hotel payment processing.
- Holding user money or operating escrow accounts.
- Issuing commercial airline/hotel tickets.
- Operating proprietary travel inventory or reservation distribution engines.
- Processing external-provider refunds or booking cancellations.
- Acting as a licensed banking entity or payment gateway.
- Guaranteeing real-time weather accuracy or continuous external API availability.
- Multi-master background conflict synchronization during offline mode.

—

7 External Interface Requirements

7.1 Authentication Interface — Google OAuth 2.0 / OIDC

- **EXT-AUTH-001:** TripNest shall support authentication through the configured Google OAuth 2.0/OIDC provider[span$_4$5]($start_span$)[$span_4$5](end_span).
- **EXT-AUTH-002:** TripNest shall receive the authenticated user's identity information from the OAuth provider[span$_4$6]($start_span$)[$span_4$6](end_span).
- **EXT-AUTH-003:** The backend shall associate the external OAuth identity with the corresponding TripNest user account in `oauth_accounts`[span$_4$7]($start_span$)[$span_4$7](end_span).
- **EXT-AUTH-004:** OAuth credentials and secrets shall remain server-side and shall not be exposed to the client[span$_4$8]($start_span$)[$span_4$8](end_span).
- **EXT-AUTH-005:** Authentication failure from the external provider shall be handled without exposing sensitive provider information.

7.2 Maps Interface — Google Maps

- **EXT-MAP-001:** TripNest shall communicate with a maps/location provider to obtain location information[span$_4$9]($start_span$)[$span_4$9](end_span).
- **EXT-MAP-002:** The system shall support location search for itinerary activities[span$_5$0]($start_span$)[$span_5$0
- **EXT-MAP-003:** The system shall be able to obtain geographic coordinates (latitude, longitude) for supported locations[span$_5$1]($start_span$)[$span_5$1](end_span).
- **EXT-MAP-004:** TripNest may use the provider for map visualization on client interfaces[span$_5$2]($start_span$
- **EXT-MAP-005:** Temporary maps-provider failures shall not prevent access to stored itinerary information[span$_5$3]($start_span$)[$span_5$3](end_span).

7.3 Weather Interface — OpenWeather

- **EXT-WEATHER-001:** TripNest shall retrieve weather information from the configured weather provider[span$_5$4]($start_span$)[$span_5$4](end_span).

- **EXT-WEATHER-002:** The system shall request weather information using relevant destination/location coordinates[span$_5$5]($start_span$)[span$_5$5](end_span).
- **EXT-WEATHER-003:** Weather responses shall be normalized into TripNest's internal representation[span
- **EXT-WEATHER-004:** Weather information may be cached using Redis with a 3-hour TTL (`weather:forecast:{lat}:{lon}`)[span$_5$7]($start_span$)[span$_5$7](end_span).
- **EXT-WEATHER-005:** The system shall handle weather-provider failures gracefully, returning null or cached fallbacks[span$_5$8]($start_span$)[span$_5$8](end_span).

7.4 Currency Exchange Interface

- **EXT-CURRENCY-001:** TripNest shall communicate with a supported exchange-rate provider[span$_5$9](*st*
- **EXT-CURRENCY-002:** The system shall retrieve exchange-rate information for supported ISO-4217 currencies[span$_6$0]($start_span$)[span$_6$0](end_span).
- **EXT-CURRENCY-003:** Exchange-rate responses shall be normalized before being consumed by financial services[span$_6$1]($start_span$)[span$_6$1](end_span).
- **EXT-CURRENCY-004:** Exchange-rate information shall be cached in Redis with a 24-hour TTL (`forex:rates:{baseCurrency}`)[span$_6$2]($start_span$)[span$_6$2](end_span).

7.5 Object Storage Interface — AWS S3

- **EXT-S3-001:** TripNest shall use AWS S3 for binary file storage[span$_6$3]($start_span$)[span$_6$3](end_span).
- **EXT-S3-002:** The backend shall generate authorized pre-signed URLs for permitted upload/-download operations (15-min TTL)[span$_6$4]($start_span$)[span$_6$4](end_span).
- **EXT-S3-003:** The backend shall store S3 object references as metadata in PostgreSQL[span$_6$5]($start_span$)[*s*
- **EXT-S3-004:** The system shall not expose unrestricted public access to private S3 objects[span$_6$6](*start*$_s$*par*
- **EXT-S3-005:** Failed uploads shall not create invalid or incomplete persistent file metadata in PostgreSQL[span$_6$7]($start_span$)[span$_6$7](end_span).

7.6 Email Interface — AWS SES

- **EXT-EMAIL-001:** TripNest shall use the configured email delivery service for supported email notifications (invites, password resets)[span$_6$8]($start_span$)[span$_6$8](end_span).
- **EXT-EMAIL-002:** The system shall generate email notifications asynchronously for supported events[span$_6$9]($start_span$)[span$_6$9](end_span).
- **EXT-EMAIL-003:** Email delivery failures shall be recorded appropriately without blocking client HTTP operations.
- **EXT-EMAIL-004:** Email credentials shall not be exposed to clients.

7.7 Redis Interface

- **EXT-REDIS-001:** TripNest shall use Redis for transient distributed data including exchange-rate caching, weather caching, and idempotency keys[span$_7$0]($start_span$)[span$_7$0](end_span).
- **EXT-REDIS-002:** Cached data shall have an appropriate expiration policy[span$_7$1]($start_span$)[span$_7$1](*en*
- **EXT-REDIS-003:** Redis failure shall not corrupt authoritative PostgreSQL data[span$_7$2]($start_span$)[span$_7$
- **EXT-REDIS-004:** Transient cache loss shall result in data retrieval/recomputation rather than permanent data loss.

7.8 PostgreSQL Interface

- **EXT-DB-001:** TripNest shall persist transactional application data in PostgreSQL[span$_7$3]($start_span$)[span
- **EXT-DB-002:** Foreign-key relationships shall maintain relational integrity[span$_7$4]($start_span$)[span$_7$4](end
- **EXT-DB-003:** Financial operations shall use appropriate database transactions (`Isolation.READ_COMMITT`
- **EXT-DB-004:** Database schema changes shall be managed through Flyway migrations[span$_7$6]($start_span$)[
- **EXT-DB-005:** The application shall not rely on Redis as the authoritative source for persistent business data[span$_7$7]($start_span$)[span$_7$7](end_span).

7.9 WebSocket/STOMP Interface

- **EXT-WS-001:** TripNest shall expose the configured WebSocket endpoint (`/ws-trip`) for real-time communication[span$_7$8]($start_span$)[span$_7$8](end_span).
- **EXT-WS-002:** Authenticated clients shall establish WebSocket connections using valid authentication credentials (Bearer JWT)[span$_7$9]($start_span$)[span$_7$9](end_span).
- **EXT-WS-003:** Trip membership shall be validated before permitting access to trip-specific topics[span$_8$0]($start_span$)[span$_8$0](end_span).
- **EXT-WS-004:** The system shall support real-time chat events on `/topic/trip/{tripId}/chat`[span$_8$1](st
- **EXT-WS-005:** The system shall support real-time itinerary update events on `/topic/trip/{tripId}/itin`
- **EXT-WS-006:** Unauthorized users shall not receive events belonging to trips they cannot access[span$_8$3]($start_span$)[span$_8$3](end_span).

7.10 REST API Interface

- **EXT-REST-001:** Protected REST endpoints shall require authentication where applicable[span$_8$4]($start_sp$
- **EXT-REST-002:** REST endpoints shall enforce contextual authorization based on trip membership and role[span$_8$5]($start_span$)[span$_8$5](end_span).
- **EXT-REST-003:** Requests shall use JSON for standard API payloads unless another content type is required[span$_8$6]($start_span$)[span$_8$6](end_span).
- **EXT-REST-004:** API errors shall follow the standardized Problem Details format (RFC 7807)[span$_8$7]($start_span$)[span$_8$7](end_span).
- **EXT-REST-005:** APIs involving financial mutations shall support the defined idempotency mechanism[span$_8$8]($start_span$)[span$_8$8](end_span).

7.11 External Interface Failure Policy

External Service	Failure Impact	Degradation Mitigation Policy
Google OAuth	Login through OAuth unavailable.	Fallback to standard email/password login.
Google Maps	Map/location features degraded.	

External Service	Failure Impact	Degradation Mitigation Policy
		Itinerary continues functioning; manual location strings permitted.
OpenWeather	Weather unavailable.	Unexpired cached forecast served from Redis; returns empty weather DTO safely[span$_8$9]($start_span$)[$span_8$9](end_span).
Currency API	Latest exchange rate unavailable.	Cached 24h Redis rate used; falls back to 1.0 base rate with UI notice[span$_9$0]($start_span$)[$span_9$0](end_span).
AWS S3	File upload/download degraded.	Metadata transaction safely blocked; returns 503 Service Unavailable[span$_9$1]($start_span$)[$span_9$1](end_span).
AWS SES	Email delivery delayed/failed.	In-app notifications persist; email jobs retried in worker queues[span$_9$2]($start_span$)[$span_9$2](end_span).
Redis	Cache/idempotency degraded.	Direct DB querying; bypass caching without blocking core logic[span$_9$3]($start_span$)[$span_9$3](end_span).
PostgreSQL	Core application unavailable.	Returns 503 via load balancer health check[span$_9$4]($start_span$)[$span_9$4](end_span).
WebSocket	Real-time features unavailable.	REST endpoints remain functional; client falls back to manual polling[span$_9$5]($start_span$)[$span_9$5](end_span).

7.12 External Interface Security

- **EXT-SEC-001:** Communication with external services shall use secure transport (TLS 1.3) where supported.
- **EXT-SEC-002:** External credentials shall be stored securely in environment variables[span$_9$6]($start_span$)[sp
- **EXT-SEC-003:** External API credentials shall never be embedded in frontend code[span$_9$7]($start_span$)[$span$

- **EXT-SEC-004:** User authorization shall be checked before generating S3 access URLs[span$_9$8]($start_span$)[s
- **EXT-SEC-005:** External responses shall be validated before being incorporated into TripNest business logic.

—

8 API Requirements & Interface Specification

8.1 API Standards

- **API-STD-001 (Base URL):** All REST APIs shall use a versioned base path: `/api/v1`[span$_9$9]($start_span$)[
- **API-STD-002 (Data Format):** REST request and response bodies shall use JSON unless another content type is explicitly required.
- **API-STD-003 (Authentication):** Protected endpoints shall require a valid access token (Bearer JWT)[span$_1$00]($start_span$)[$span_1$00](end_span).
- **API-STD-004 (Authorization):** Authentication shall not automatically grant access to a resource. The backend shall verify the user's relationship with the requested trip/resource[span$_1$01]($start_span$)[
- **API-STD-005 (Error Format):** API errors shall use the standardized Problem Details format (RFC 7807)[span$_1$02]($start_span$)[$span_1$02](end_span).
- **API-STD-006 (Pagination):** Large collections shall support pagination (`?page=0&size=20`)[span$_1$03](*sta*
- **API-STD-007 (Idempotency):** Financial mutation endpoints shall support the defined `Idempotency-Key` mechanism[span$_1$04]($start_span$)[$span_1$04](end_span).

8.2 REST API Endpoint Specifications

8.2.1 1. Authentication APIs (`/api/v1/auth`)

- **POST `/api/v1/auth/register`:** Creates a new account. Body: {`"name": "..."`, `"email": "..."`, `"password": "..."`} → Returns 201 Created[span$_1$05]($start_span$)[$span_1$05](end_span).
- **POST `/api/v1/auth/login`:** Authenticates user credentials → Returns 200 OK with access token and refresh token[span$_1$06]($start_span$)[$span_1$06](end_span).
- **POST `/api/v1/auth/refresh`:** Generates a new access token using a valid refresh token → Returns 200 OK[span$_1$07]($start_span$)[$span_1$07](end_span).
- **POST `/api/v1/auth/logout`:** Invalidates active refresh token in PostgreSQL → Returns 204 No Content[span$_1$08]($start_span$)[$span_1$08](end_span).
- **POST `/api/v1/auth/forgot-password`:** Initiates password recovery email → Returns 200 OK.
- **POST `/api/v1/auth/reset-password`:** Resets password via recovery token → Returns 200 OK.

8.2.2 2. User Profile APIs (`/api/v1/users`)

- **GET `/api/v1/users/me`:** Returns authenticated user's profile → Returns 200 OK[span$_1$09]($start_span$)[*span*
- **PATCH `/api/v1/users/me`:** Updates user's profile details → Returns 200 OK[span$_1$10]($start_span$)[$span_1$10
- **DELETE `/api/v1/users/me`:** Initiates account deactivation/anonymization → Returns 204 No Content[span$_1$11]($start_span$)[$span_1$11](end_span).

8.2.3 3. Trip Workspace APIs (/api/v1/trips)

- **POST /api/v1/trips:** Atomically creates trip and adds creator as OWNER → Returns 201 Created[span$_1$12]($start_span$)[$span_1$12](end_span).
- **GET /api/v1/trips:** Returns paginated trips accessible to user (?page=0&size=20) → Returns 200 OK.
- **GET /api/v1/trips/{tripId}:** Returns trip workspace details → Returns 200 OK[span$_1$13]($start_span$)[sp

- **PATCH /api/v1/trips/{tripId}:** Updates trip metadata (Req: role >= ADMIN) → Returns 200 OK[span$_1$14]($start_span$)[$span_1$14](end_span).
- **DELETE /api/v1/trips/{tripId}:** Deletes trip workspace (Req: role == OWNER) → Returns 204 No Content[span$_1$15]($start_span$)[$span_1$15](end_span).

8.2.4 4. Membership APIs (/api/v1/trips/{tripId}/members)

- **GET /api/v1/trips/{tripId}/members:** Returns trip member roster → Returns 200 OK[span$_1$16]($start_sp$

- **POST /api/v1/trips/{tripId}/invitations:** Creates invite token (Req: role >= ADMIN) → Returns 201 Created[span$_1$17]($start_span$)[$span_1$17](end_span).
- **POST /api/v1/invitations/{token}/accept:** Accepts an invitation → Returns 200 OK[span$_1$18]($start_sp$

- **DELETE /api/v1/trips/{tripId}/members/{userId}:** Removes member (Req: role >= ADMIN) → Returns 204 No Content[span$_1$19]($start_span$)[$span_1$19](end_span).
- **POST /api/v1/trips/{tripId}/leave:** Allows current user to leave workspace → Returns 204 No Content[span$_1$20]($start_span$)[$span_1$20](end_span).
- **PATCH /api/v1/trips/{tripId}/members/{userId}/role:** Modifies member role → Returns 200 OK[span$_1$21]($start_span$)[$span_1$21](end_span).
- **POST /api/v1/trips/{tripId}/transfer-ownership:** Transfers root ownership → Returns 200 OK[span$_1$22]($start_span$)[$span_1$22](end_span).

8.2.5 5. Itinerary APIs (/api/v1/trips/{tripId}/itinerary)

- **GET /api/v1/trips/{tripId}/itinerary:** Returns complete itinerary structure → Returns 200 OK.
- **POST /api/v1/trips/{tripId}/days:** Creates an itinerary day → Returns 201 Created[span$_1$23]($start_spa$

- **PATCH /api/v1/trips/{tripId}/days/{dayId}:** Updates day title/notes → Returns 200 OK[span$_1$24]($start_span$)[$span_1$24](end_span).
- **DELETE /api/v1/trips/{tripId}/days/{dayId}:** Deletes day container → Returns 204 No Content[span$_1$25]($start_span$)[$span_1$25](end_span).
- **POST /api/v1/trips/{tripId}/days/{dayId}/activities:** Adds activity stop → Returns 201 Created[span$_1$26]($start_span$)[$span_1$26](end_span).
- **PATCH /api/v1/activities/{activityId}:** Updates activity with version check ({"version": 4, ...}) → Returns 200 OK or 409 Conflict[span$_1$27]($start_span$)[$span_1$27](end_span).
- **DELETE /api/v1/activities/{activityId}:** Deletes activity stop → Returns 204 No Content[span$_1$28](

8.2.6 6. Expense & Settlement APIs

- **GET /api/v1/trips/{tripId}/expenses:** Returns paginated expenses → Returns 200 OK.
- **POST /api/v1/trips/{tripId}/expenses:** Transactionally creates expense and splits with Idempotency-Key → Returns 201 Created[span$_1$29]($start_span$)[$span_1$29](end_span).
- **PATCH /api/v1/expenses/{expenseId}:** Updates expense details → Returns 200 OK[span$_1$30]($start_span$

- **DELETE** `/api/v1/expenses/{expenseId}`: Deletes expense and recalculates balances → Returns 204 No Content[span$_1$31]($start_span$)[span$_1$31](end_span).
- **GET** `/api/v1/trips/{tripId}/balances`: Returns current pairwise net balance matrix → Returns 200 OK[span$_1$32]($start_span$)[span$_1$32](end_span).
- **GET** `/api/v1/trips/{tripId}/settlements`: Returns settlement history → Returns 200 OK[span$_1$33]($start_span$)[span$_1$33](end_span).
- **POST** `/api/v1/trips/{tripId}/settlements`: Records debt clearance payment with `Idempotency-Key` → Returns 201 Created[span$_1$34]($start_span$)[span$_1$34](end_span).
- **GET** `/api/v1/trips/{tripId}/settlements/suggestions`: Returns simplified debt reduction graph → Returns 200 OK[span$_1$35]($start_span$)[span$_1$35](end_span).

8.2.7 7. Travel Wallet APIs (`/api/v1/wallet`)

- **GET** `/api/v1/wallet`: Returns caller's private wallet items → Returns 200 OK.
- **GET** `/api/v1/wallet/items/{walletItemId}`: Returns authorized wallet item → Returns 200 OK.
- **POST** `/api/v1/wallet/items`: Records personal wallet item → Returns 201 Created.
- **PATCH** `/api/v1/wallet/items/{walletItemId}`: Updates wallet item metadata → Returns 200 OK.
- **DELETE** `/api/v1/wallet/items/{walletItemId}`: Deletes wallet item → Returns 204 No Content.
- **GET** `/api/v1/trips/{tripId}/wallet-items`: Returns shared wallet items explicitly linked to trip → Returns 200 OK.

8.2.8 8. Document & Media S3 APIs

- **POST** `/api/v1/trips/{tripId}/documents/upload-url`: Generates 15-min pre-signed upload URL → Returns 200 OK[span$_1$36]($start_span$)[span$_1$36](end_span).
- **POST** `/api/v1/trips/{tripId}/documents`: Commits document metadata after S3 upload → Returns 201 Created[span$_1$37]($start_span$)[span$_1$37](end_span).
- **GET** `/api/v1/trips/{tripId}/documents`: Returns accessible document records → Returns 200 OK[span$_1$38]($start_span$)[span$_1$38](end_span).
- **GET** `/api/v1/documents/{documentId}/download-url`: Generates temporary download URL → Returns 200 OK[span$_1$39]($start_span$)[span$_1$39](end_span).
- **DELETE** `/api/v1/documents/{documentId}`: Deletes document metadata → Returns 204 No Content[span$_1$40]($start_span$)[span$_1$40](end_span).
- **POST** `/api/v1/trips/{tripId}/media/upload-url`: Generates pre-signed media upload URL → Returns 200 OK[span$_1$41]($start_span$)[span$_1$41](end_span).
- **POST** `/api/v1/trips/{tripId}/media`: Creates media metadata → Returns 201 Created[span$_1$42]($start_s$
- **GET** `/api/v1/trips/{tripId}/media`: Returns paginated trip media → Returns 200 OK[span$_1$43]($start_s$
- **PATCH** `/api/v1/media/{mediaId}`: Updates media caption → Returns 200 OK[span$_1$44]($start_span$)[span
- **DELETE** `/api/v1/media/{mediaId}`: Deletes media metadata → Returns 204 No Content[span$_1$45]($start_s$

8.2.9 9. Chat & Notification APIs

- **GET** `/api/v1/trips/{tripId}/messages`: Returns paginated chat message history → Returns 200 OK[span$_1$46]($start_span$)[span$_1$46](end_span).
- **DELETE** `/api/v1/messages/{messageId}`: Moderates/deletes chat message → Returns 204 No Content.

- **GET `/api/v1/notifications`:** Returns current user's paginated notifications → Returns 200 OK[span$_1$47]($start_span$)[span$_1$47](end_span).
- **PATCH `/api/v1/notifications/{id}/read`:** Marks a notification as read → Returns 200 OK[span$_1$48]($start_span$)[span$_1$48](end_span).
- **POST `/api/v1/notifications/read-all`:** Marks all user notifications as read → Returns 204 No Content.

8.2.10 10. External Proxy Data APIs

- **GET `/api/v1/weather?lat={lat}&lon={lon}`:** Returns Redis cached destination weather → Returns 200 OK[span$_1$49]($start_span$)[span$_1$49](end_span).
- **GET `/api/v1/locations/search?q={query}`:** Proxies Google Places autocomplete → Returns 200 OK[span$_1$50]($start_span$)[span$_1$50](end_span).
- **GET `/api/v1/currency/rates?base={base}`:** Returns Redis cached forex conversion rates → Returns 200 OK[span$_1$51]($start_span$)[span$_1$51](end_span).

8.3 WebSocket STOMP Channel Topology

- **Handshake Endpoint:** `wss://api.tripnest.com/ws-trip`[span$_1$52]($start_span$)[span$_1$52](end_span).
- **Client → Server Channel:** `/app/trip/{tripId}/chat` (Sends chat frame)[span$_1$53]($start_span$)[span$_1$53](
- **Broadcast Topics:**
  - `/topic/trip/{tripId}/chat` — Live chat delivery to members[span$_1$54]($start_span$)[span$_1$54](end_span
  - `/topic/trip/{tripId}/itinerary` — Activity reordering and mutation alerts[span$_1$55]($start_span$)[sp
- **User Private Queue:** `/user/queue/notifications` — Direct write-then-broadcast alert frames[span$_1$56]($start_span$)[span$_1$56](end_span).

—

9 ER Diagram & Database Specification

9.1 Cardinality Summary

Entity Relationship Relationship Purpose	Cardinality
`users` → `oauth_accounts` External OAuth provider claims mapped to user[span$_1$57]($start_span$)[span$_1$57](end_span).	$1 : N$
`users` → `refresh_tokens` Multi-device token management with individual revocation[span$_1$58]($start_span$)[span$_1$58](end_span).	$1 : N$
`users` → `trips (owner)` Root ownership and administrative authority[span$_1$59]($start_span$)[span$_1$59](end_span).	$1 : N$
`users` ↔ `trips`	$M : N$ via `trip_members`

Entity Relationship	Cardinality	Relationship Purpose
`trips` → `trip_invitations`	$1:N$	Resolves trip membership with assigned role[span$_1$60]($start_span$)[$span_1$60](end_span).
`trips` → `itinerary_days`	$1:N$	Tokenized, expiring membership invite links[span$_1$61]($start_span$)[$span_1$61](end_span).
`itinerary_days` → `activities`	$1:N$	Calendar day scheduling containers[span$_1$62]($start_span$)[$span_1$62](end_span).
`trips` → `expenses`	$1:N$	Activity stops sequenced via LexoRank[span$_1$63]($start_span$)[$span_1$63](end_span).
`expenses` → `expense_splits`	$1:N$	Group financial expenditure records[span$_1$64]($start_span$)[$span_1$64](end_span).
`users` → `expense_splits`	$1:N$	Itemized debt allocations per member[span$_1$65]($start_span$)[$span_1$65](end_span).
`trips` → `settlements`	$1:N$	Direct debtor attribution for balance tracking[span$_1$66]($start_span$)[$span_1$66](end_span).
`users` → `wallet_items`	$1:N$	Pairwise debt clearance transactions[span$_1$67]($start_span$)[$span_1$67](end_span).
`trips` → `wallet_items`	$1:N$ (Optional)	Private travel credentials owned by user[span$_1$68]($start_span$)[$span_1$68](end_span).
`trips` → `documents`	$1:N$	Optional linkage of wallet assets to trip[span$_1$69]($start_span$)[$span_1$69](end_span).
`trips` → `trip_media`	$1:N$	Shared trip document metadata in S3[span$_1$70]($start_span$)[$span_1$70](end_span).
`trips` → `chat_messages`	$1:N$	Shared photos and videos in trip gallery[span$_1$71]($start_span$)[$span_1$71](end_span).
`users` → `chat_messages`	$1:N$	Persistent group chat message history[span$_1$72]($start_span$)[$span_1$72](end_span).
`users` → `notifications`	$1:N$	Author attribution for chat messages[span$_1$73]($start_span$)[$span_1$73](end_span).
`trips` / `users` → `audit_logs`	$1:N$	Targeted in-app alerts dispatched via STOMP[span$_1$74]($start_span$)[$span_1$74](end_span).

Entity Relationship Relationship Purpose	Cardinality
High-impact security and financial audit trail[span$_1$75]($start_span$)[$span_1$75](end_span).	

9.2 Database Indexing Strategy

Target Table Access Pattern & Optimization Goal	Indexed Column(s)
`users` $O(1)$ fast credential lookup during authentication[span$_1$76]($start_span$)[$span_1$76](end_span).	`email` (UNIQUE)
`trip_members` Contextual RBAC authorization evaluation on every request[span$_1$77]($start_span$)[$span_1$77](end_span).	`(trip_id, user_id)` (UNIQUE)
`trip_members` Retrieval of all workspaces joined by user[span$_1$78]($start_span$)[$span_1$78](end_span).	`user_id`
`trip_invitations` Rapid token lookup when claiming invite links[span$_1$79]($start_span$)[$span_1$79](end_span).	`token_hash` (UNIQUE)
`trip_invitations` Fetching pending invitations for a workspace[span$_1$80]($start_span$)[$span_1$80](end_span).	`trip_id`
`itinerary_days` Day ordering retrieval within a trip[span$_1$81]($start_span$)[$span_1$81](end_span).	`(trip_id, day_number)` (UNIQUE)
`activities` Drag-and-drop LexoRank sequential sorting within a day[span$_1$82]($start_span$)[$span_1$82](end_span).	`(day_id, order_index)`
`expenses` Paginated chronological expense feed retrieval[span$_1$83]($start_span$)[$span_1$83](end_span).	`(trip_id, expense_date DESC)`
`expenses`	`payer_id`

Target Table Access Pattern & Optimization Goal	Indexed Column(s)
Aggregation of total expenses paid per member[span$_1$84]($start_span$)[$span_1$84](end_span).	
`expense_splits` Fetching split allocations for a specific expense[span$_1$85]($start_span$)[$span_1$85](end_span).	`expense_id`
`expense_splits` Rapid balance aggregation across all member splits[span$_1$86]($start_span$)[$span_1$86](end_span).	`user_id`
`settlements` Settlement history and balance recalculation[span$_1$87]($start_span$)[$span_1$87](end_span).	`trip_id`
`wallet_items` User private Travel Wallet loading[span$_1$88]($start_span$)[$span_1$88](end_span).	`user_id`
`wallet_items` Trip-associated wallet item filtering[span$_1$89]($start_span$)[$span_1$89](end_span).	`trip_id`
`documents` Shared trip document listing[span$_1$90]($start_span$)[$span_1$90](end_span).	`trip_id`
`trip_media` Shared gallery carousel loading[span$_1$91]($start_span$)[$span_1$91](end_span).	`trip_id`
`chat_messages` Paginated live chat message history retrieval[span$_1$92]($start_span$)[$span_1$92](end_span).	`(trip_id, created_at DESC)`
`notifications` Unread notification bell count and feed loading[span$_1$93]($start_span$)[$span_1$93](end_span).	`(user_id, is_read, created_at DESC)`
`audit_logs` Trip audit trail inspection for admins/owners[span$_1$94]($start_span$)[$span_1$94](end_span).	`(trip_id, created_at DESC)`
`audit_logs` User-specific action history[span$_1$95]($start_span$)[$span_1$95](end_span).	`(actor_id, created_at DESC)`

9.3 Cascade Deletion & Data Retention Policy

- **Trip Workspace Deletion:** Deleting a trip cascades to all operational children:

  trips $\xrightarrow{\text{CASCADE}}$ {trip_members, trip_invitations, itinerary_days, expenses, settlements, documents, trip_media,

- **Wallet Preservation:** Referenced `wallet_items` have their `trip_id` set to `NULL` via `ON DELETE SET NULL`, preserving the user's private records[span$_1$97]($start_span$)[$span_1$97](end_span).
- **User Deletion / Soft-Anonymization:** Foreign keys on `expenses.payer_id`, `expense_splits.user_id`, and `settlements` enforce `ON DELETE RESTRICT`[span$_1$98]($start_span$)[$span_1$98](end_span).$Usersaresoft-deleted/anonymized$(`full_name = 'Former Member'`, $email/hashpurged$)$tomaintainfinancialledgerconsi$

—

10 System Architecture & Component Design

10.1 Architectural Style: Modular Monolith

TripNest uses a Domain-Driven Modular Monolith architecture on Spring Boot 3.x[span$_2$00]($start_span$)[$span_2$00](e

10.2 Backend Package-by-Feature Layout

```
com.tripnest
|-- auth                    // Registration, Login, Google OAuth, JWT, Tokens
|-- trip                    // Trip Workspace, Membership, Contextual RBAC, Invites
|-- itinerary               // Days, Activities, LexoRank Ordering, Version Lock
|-- expense                 // Multi-Currency Expenses, Splits, Debt Simplification
|-- wallet                  // Private Travel Wallet Documents
|-- media                   // Shared Photo & Video Gallery Metadata
|-- chat                    // Persistent Live Messaging (STOMP)
|-- notification            // Write-Then-Broadcast In-App Alerts
|-- audit                   // Immutable Audit Event Log Ingestion
|-- security                // SecurityFilterChain, JwtFilter, RBAC Evaluators
|-- integration             // External Adapters (Maps, Weather, Currency, AWS S3,
   SES)
`-- common                  // GlobalExceptionHandler, RFC 7807, Utilities
```

—

11 Critical Sequence Diagrams & Workflows

11.1 11.1 Create Trip & Assign Owner

1. Client issues `POST /api/v1/trips` with Bearer JWT[span$_2$01]($start_span$)[$span_2$01](end_span).
1. Spring opens `@Transactional` boundary[span$_2$02]($start_span$)[$span_2$02](end_span).
1. Inserts new record into `trips`[span$_2$03]($start_span$)[$span_2$03](end_span).
1. Inserts record into `trip_members` with `role = 'OWNER'`[span$_2$04]($start_span$)[$span_2$04](end_span).

1. Appends `TRIP_CREATED` event to `audit_logs`[span$_2$05]($start_span$)[$span_2$05](end_span).
1. Commits transaction atomically and returns created Trip Workspace DTO[span$_2$06]($start_span$)[$span_2$06](en

11.2 11.2 Collaborative Itinerary Update & Optimistic Locking

1. User A submits `PATCH /api/v1/activities/{id}` with updated details and current `version`[span$_2$07]($start$

1. Backend checks contextual permissions (`role >= MEMBER`)[span$_2$08]($start_span$)[$span_2$08](end_span).

1. Database validates supplied version against stored entity version[span$_2$09]($start_span$)[$span_2$09](end_span).

1. If versions match: updates activity, increments version, commits transaction, and publishes event to STOMP topic `/topic/trip/{tripId}/itinerary`[span$_2$10]($start_span$)[$span_2$10](end_span).

1. If version mismatch: aborts immediately with `409 Conflict` to prevent silent data overwrites[span$_2$11]($start_s$

11.3 11.3 Idempotent Expense Logging & Split Allocation

1. Client issues `POST /api/v1/trips/{tripId}/expenses` with `Idempotency-Key` header[span$_2$12]($start_span$)

1. Backend queries Redis cache for existing idempotency key[span$_2$13]($start_span$)[$span_2$13](end_span).

1. If key absent: validates split sum ($\sum$Splits = Amount), begins transaction, persists expense, batch inserts splits, commits, caches key in Redis (24h TTL), and pushes notifications via STOMP[span$_2$14]($start_span$)[$span_2$14](end_span).
1. If key present: returns previously cached HTTP response without re-executing database mutations[span$_2$15](s

11.4 11.4 Two-Phase Direct-to-S3 Upload

1. Client requests pre-signed URL via `POST /api/v1/trips/{tripId}/documents/upload-url`[span$_2$16]($start$

1. Backend authorizes request and generates a 15-minute expiring S3 pre-signed `PUT` URL[span$_2$17]($start_span$)[s

1. Client streams binary payload directly to AWS S3 bucket[span$_2$18]($start_span$)[$span_2$18](end_span).

1. Client calls `POST /api/v1/trips/{tripId}/documents` to commit metadata to PostgreSQL[span$_2$19]($start_s$

—

12 Security Specification

12.1 Authentication & Token Management

- **SEC-001:** All protected APIs require a valid authenticated session (BCrypt hashed passwords or Google OAuth/OIDC)[span$_2$20]($start_span$)[$span_2$20](end_span).
- **SEC-002:** Access tokens are stateless HMAC-SHA256 signed JWTs (15-min TTL); refresh tokens (7-day TTL) are stored as hashes in PostgreSQL and support instant revocation[span$_2$21]($start_span$)[spa

- **SEC-003:** JWT signing secrets and OAuth client credentials shall never be exposed to frontend code[span$_2$22]($start_span$)[$span_2$22](end_span).

12.2 Contextual Role-Based Access Control (RBAC)

Authentication alone does not grant access. The security engine evaluates the contextual chain:

Authenticated User ⟶ Target Resource ⟶ Trip ⟶ Membership ⟶ Role ⟶ Permission Gate[span$_2$23]($start$

OWNER: Full trip management, member removal, ownership transfer, trip deletion[span$_2$24]($start_span$)[$span_2$24]

ADMIN: Moderate chat, manage shared files, invite members, modify itinerary[span$_2$25]($start_span$)[$span_2$25](en

MEMBER: Add/reorder activities, log expenses and splits, upload media, post chat[span$_2$26]($start_span$)[$span_2$26

VIEWER: Read-only access to itinerary, media, documents, and balances[span$_2$27]($start_span$)[$span_2$27](end_span

12.3 Real-Time & Storage Security

- **SEC-004:** A STOMP `ChannelInterceptor` validates JWT tokens and trip membership before authorizing topic subscriptions[span$_2$28]($start_span$)[$span_2$28](end_span).
- **SEC-005:** S3 buckets block all public read access; downloads mandate 15-minute pre-signed `GET` URLs[span$_2$29]($start_span$)[$span_2$29](end_span).
- **SEC-006:** Sensitive endpoints enforce rate limiting via Bucket4j algorithms[span$_2$30]($start_span$)[$span_2$30](e

- **SEC-007:** High-impact mutations produce immutable audit logs (`TRIP_CREATED`, `MEMBER_REMOVED`, `ROLE_CHANGED`, `EXPENSE_CREATED`)[span$_2$31]($start_span$)[$span_2$31](end_span).

—

13 Testing & Acceptance Criteria

13.1 Testing Verification Matrix

- **Unit Testing:** Debt simplification algorithm, split sum validations, LexoRank reordering logic, currency math[span$_2$32]($start_span$)[$span_2$32](end_span).
- **Integration Testing:** Spring Data JPA persistence, Redis TTL cache expiry, Flyway migrations, S3 pre-signing beans[span$_2$33]($start_span$)[$span_2$33](end_span).
- **Financial Integrity:** Verification that $\sum$ Splits = Total Expense and balance matrices sum to zero ($\sum B_u = 0$)[span$_2$34]($start_span$)[$span_2$34](end_span).
- **Concurrency Testing:** Validating that concurrent activity updates with identical `@Version` yield 1 success and 1 `409 Conflict`[span$_2$35]($start_span$)[$span_2$35](end_span).
- **Idempotency Testing:** Verifying duplicate POST requests with the same `Idempotency-Key` return identical cached results without duplicate database inserts[span$_2$36]($start_span$)[$span_2$36](end_span).

- **Fault Tolerance:** Simulating 500 errors/timeouts on external weather and currency APIs to verify non-blocking graceful fallback[span$_2$37]($start_span$)[$span_2$37](end_span).

13.2 Definition of Done (DoD)

A backend feature is accepted as complete only when:

Requirements Verified ⟶ Contextual RBAC Guarded ⟶ Transactional DB Layer ⟶ Unit/Integration Tests P